Collab Specific Imports:

```
!pip install -q -U transformers peft accelerate datasets torchao tqdm
import sys
from google.colab import drive

drive.mount('/content/drive')
sys.path.append('/content/drive/MyDrive/final-project')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remo
```

Library and File Imports:

```
import torch
import matplotlib.pyplot as plt
from google.colab import drive, files
from accelerate import Accelerator
from peft import PeftModel
from transformers import get_cosine_schedule_with_warmup, AutoModelForCausalLM
from models.model_builder import load_sft_model_and_tokenizer, load_dpo_model_and_tokenizer
from data.data_loader import load_sft_data, load_dpo_data
from training.dpo import train_dpo, test_dpo
from training.sft import train_sft, test_sft
from utils.visualize import plot_dpo_metrics, plot_sft_metrics
from utils.inference import generate_and_print_example_outputs
```

```
WARNING:torchao:Skipping import of cpp extensions due to incompatible torch version. Please upgrade to torch >= 2.11.0
```

Load SFT model and tokenizer and SFT data:

```
sft_model, sft_tokenizer = load_sft_model_and_tokenizer()
sft_train_dataloader, sft_test_dataloader, sft_test_sample = load_sft_data(sft_tokenizer, 10) # optionally adjust bat
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits an
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_T
Loading weights: 100%  290/290 [00:02<00:00, 129.82it/s]
```

```
print(sft_model.device)
```

```
cuda:0
```

Set up parameters for training and perform supervised finetuning on the model:

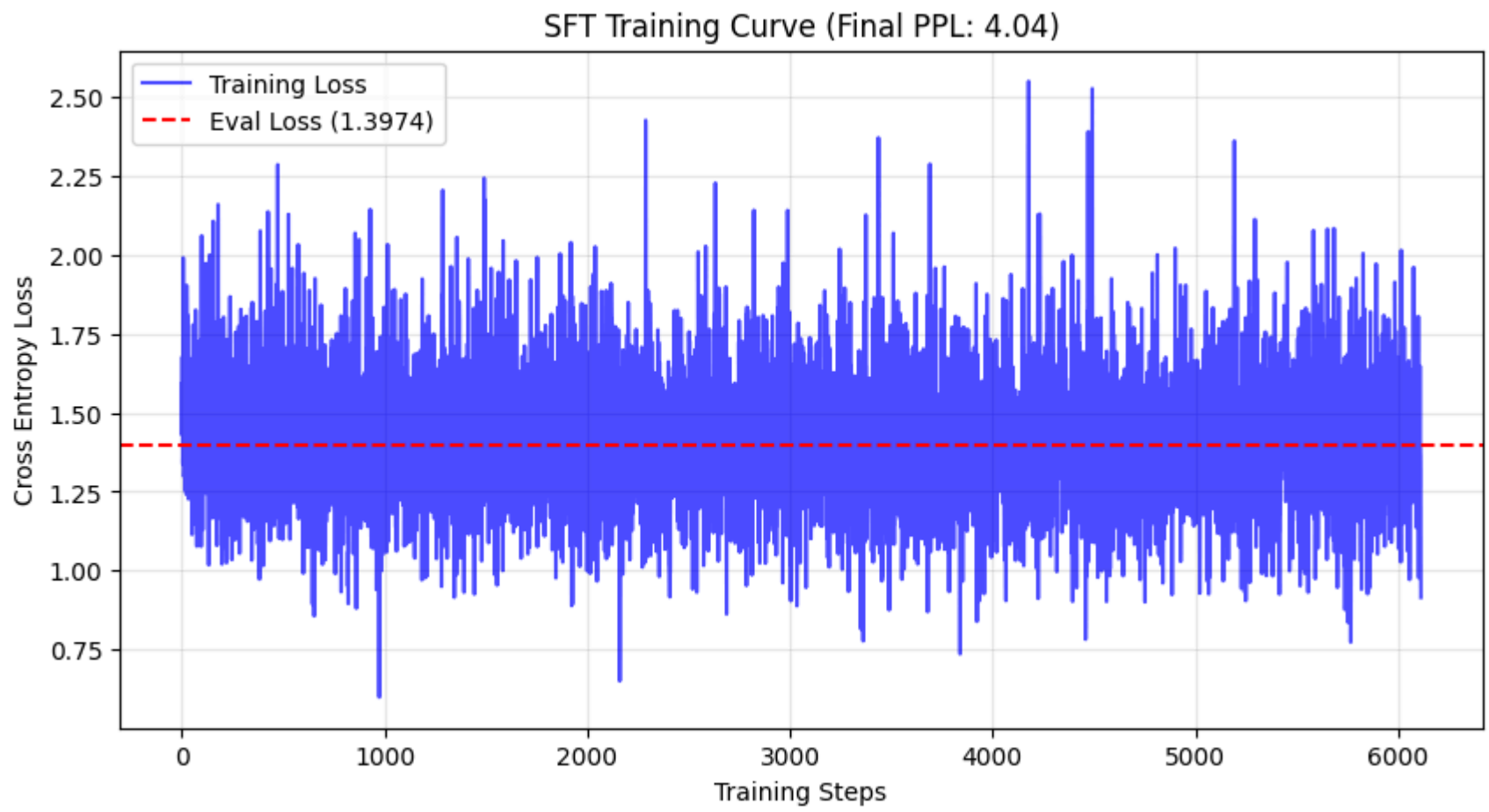
```
# optimizer, scheduler, epochs, accelerator
epochs = 1
lr = 2e-4
weight_decay = 0.01
accelerator = Accelerator(gradient_accumulation_steps=3, mixed_precision="fp16")
optimizer = torch.optim.AdamW(params=[p for p in sft_model.parameters() if p.requires_grad], lr=lr, weight_dec

total_updates = (len(sft_train_dataloader) // accelerator.gradient_accumulation_steps) * epochs
warmup_steps = int(0.1 * total_updates)
scheduler = get_cosine_schedule_with_warmup(optimizer, warmup_steps, total_updates)

sft_model, optimizer, sft_train_dataloader, scheduler = accelerator.prepare(sft_model, optimizer, sft_train_da

sft_step_losses = train_sft(sft_model, sft_train_dataloader, optimizer, scheduler, accelerator, epochs)
```

```
DPO Epoch 1/1: 100%  6114/6114 [50:18<00:00,  2.26it/s, loss=0.9148]
Average epoch loss for epoch 0 : 1.412366267879843
```

Test and experiment with the sft network :

```
# Calculate Test Metrics
sft_test_accuracy, sft_test_loss, sft_test_perplexity = test_sft(sft_model, sft_test_dataloader)
print(f"SFT Test Accuracy: {sft_test_accuracy}")
print(f"SFT Test Loss: {sft_test_loss}")
print(f"SFT Test Perplexity: {sft_test_perplexity}")

# Visualize training with final test metrics
plot_sft_metrics(sft_step_losses, sft_test_loss, sft_test_perplexity)

# Generate example input/outputs
generate_and_print_example_outputs(sft_model, sft_tokenizer, sft_test_sample)
```

```
SFT Test Accuracy: 0.6446132075471698
SFT Test Loss: 1.397405144572258
SFT Test Perplexity: 4.04469108581543
```




```
==================================================
PROMPT:
Write an investigative report on the state of homelessness in America.

MODEL OUTPUT:
Title: The State of Homelessness in America: A Comprehensive Analysis

Introduction

Homelessness is a pervasive issue in the United States, affecting millions of people each year. Despite the country's

==================================================
PROMPT:
A factory produces two types of toys: robots and cars. The factory can produce 50 robots and 100 cars per day. The dem
What is the equilibrium price and quantity for each toy? How would a decrease in the cost of production affect the equ

MODEL OUTPUT:
To find the equilibrium price and quantity for each toy, we need to set up the following equations:

1. For robots:
   p = 10 - 0.05q


==================================================
PROMPT:
Detailed Instructions: In this task, you will be given a sentence, followed by ", so". You should complete the given t
Q: The teacher took roll, so
A:

MODEL OUTPUT:
The teacher took roll, so the students were able to roll their own.
Confidence: 90%
```

Save the finetuned model :

```
accelerator.wait_for_everyone()
unwrapped_sft_model = accelerator.unwrap_model(sft_model)
unwrapped_sft_model.save_pretrained('sft_adapter')
sft_tokenizer.save_pretrained('sft_adapter')
print("successfully saved sft adapter and tokenizer")

print("loading base model and integrating with sft adapter")
base_model = AutoModelForCausalLM.from_pretrained('Qwen/Qwen2-0.5B', torch_dtype=torch.bfloat16, device_map='auto')
sft_peft_model = PeftModel.from_pretrained(base_model, 'sft_adapter')
merged_model = sft_peft_model.merge_and_unload()
```

```
merged_model.save_pretrained('merged_sft_model')
sft_tokenizer.save_pretrained('merged_sft_model')
```

```
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
successfully saved sft adapter and tokenizer
loading base model and integrating with sft adapter
```

Loading weights: 100% 290/290 [00:00<00:00, 691.45it/s]

Writing model shards: 100% 1/1 [00:07<00:00, 7.64s/it]

```
('merged_sft_model/tokenizer_config.json',
 'merged_sft_model/chat_template.jinja',
 'merged_sft_model/tokenizer.json')
```

Clear Cache so don't run into out of memory issues:

```
import gc
gc.collect()
torch.cuda.empty_cache()
```

```
# Copy the folders into mounted Drive
!cp -r sft_adapter /content/drive/MyDrive/final-project/
!cp -r merged_sft_model /content/drive/MyDrive/final-project/
```

Load in the dpo model and data :

```
print("Loading DPO model")
dpo_model, dpo_tokenizer = load_dpo_model_and_tokenizer('merged_sft_model')
dpo_train_dataloader, dpo_test_dataloader, dpo_test_sample = load_dpo_data(dpo_tokenizer, 5) # batch size defaults to
```

```
Loading DPO model
```

Loading weights: 100% 290/290 [00:01<00:00, 212.68it/s]

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits an
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_T
```

Set up hyperparameters and train the dpo model :

```
dpo_epochs = 1
dpo_lr = 1e-5
dpo_beta = 0.1
dpo_weight_decay = 0.01
dpo_accelerator = Accelerator(gradient_accumulation_steps=6, mixed_precision='fp16')
dpo_optimizer = torch.optim.AdamW([p for p in dpo_model.parameters() if p.requires_grad], dpo_lr, weight_decay=dpo_we

dpo_total_updates = (len(dpo_train_dataloader) // dpo_accelerator.gradient_accumulation_steps) * dpo_epochs
dpo_warmup_steps = int(0.1 * dpo_total_updates)
dpo_scheduler = get_cosine_schedule_with_warmup(dpo_optimizer, dpo_warmup_steps, dpo_total_updates)

dpo_model, dpo_optimizer, dpo_train_dataloader, dpo_scheduler = dpo_accelerator.prepare(dpo_model, dpo_optimizer, dpo
dpo_step_losses, dpo_step_rewards = train_dpo(dpo_model, dpo_train_dataloader, dpo_optimizer, dpo_scheduler, dpo_acce
```

DPO Epoch 1/1: 100% 12227/12227 [2:31:51<00:00, 1.33it/s, loss=0.8453, reward=-0.2452]

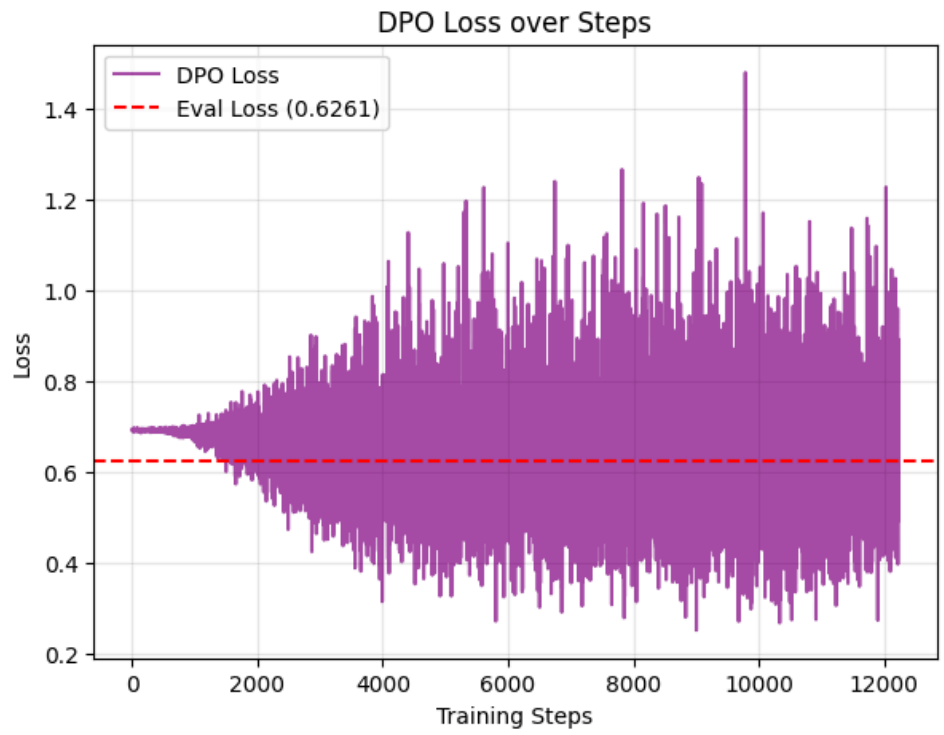
```
Average epoch loss for epoch 0 : 0.6432746284443057
```

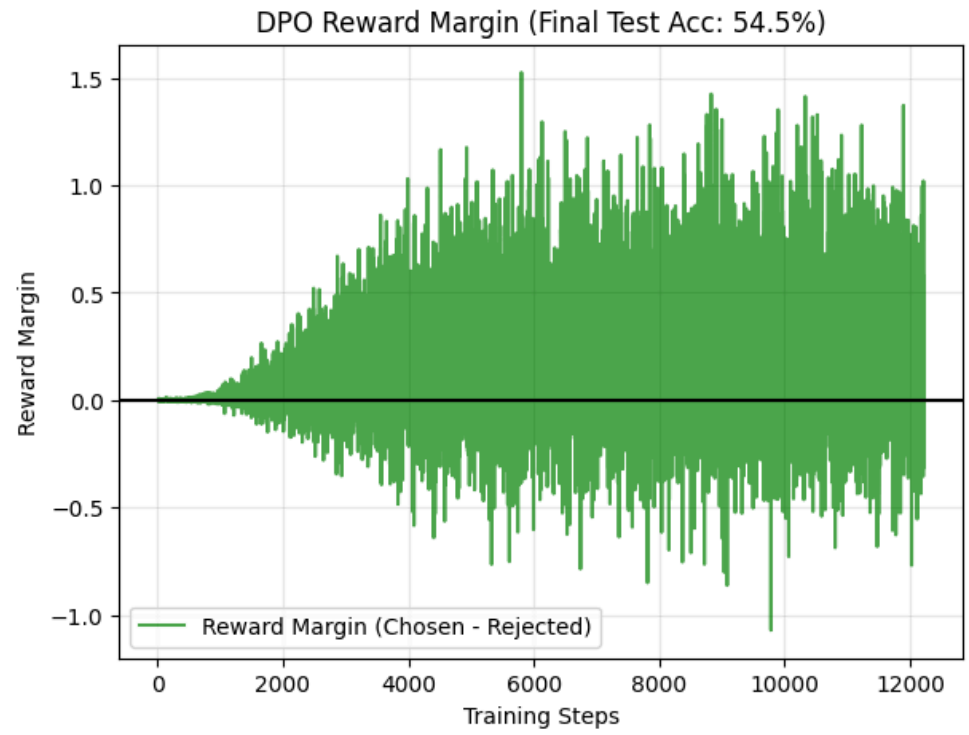
Test and experiment with DPO

```
# Calculate Test Metrics
dpo_test_accuracy, dpo_test_loss = test_dpo(dpo_model, dpo_test_dataloader)
print(f"DPO test accuracy : {dpo_test_accuracy}")
print(f"DPO test loss: {dpo_test_loss}")

# Visualize training with final test metrics
plot_dpo_metrics(dpo_step_losses, dpo_step_rewards, dpo_test_accuracy, dpo_test_loss)

# Generate example input/outputs
generate_and_print_example_outputs(dpo_model, dpo_tokenizer, dpo_test_sample)
```

DPO Testing: 100% 400/400 [03:03<00:00, 2.24it/s]

```
DPO test accuracy : 0.5445000126212836
DPO test loss: 0.6260578994452953
```






```
==================================================
PROMPT:
Write an investigative report on the state of homelessness in America.

MODEL OUTPUT:
Title: The State of Homelessness in America: A Comprehensive Analysis

Introduction

Homelessness is a pervasive issue in the United States, affecting millions of individuals each year. Despite the cour

==================================================
PROMPT:
A factory produces two types of toys: robots and cars. The factory can produce 50 robots and 100 cars per day. The de
What is the equilibrium price and quantity for each toy? How would a decrease in the cost of production affect the ec

MODEL OUTPUT:
Let's first find the equilibrium prices and quantities for each toy:

1. Equilibrium price for robots:
The factory can produce 50 robots per day, so the equilibrium price for robots is:


==================================================
PROMPT:
Detailed Instructions: In this task, you will be given a sentence, followed by ", so". You should complete the given
Q: The teacher took roll, so
A:

MODEL OUTPUT:
The teacher rolled the dice, so the result was a 6.
Confidence: 95%
Confidence: 95%
Confidence: 95
==================================================
PROMPT:
```

Save the Trained DPO Adapters

```
dpo_accelerator.wait_for_everyone()
unwrapped_dpo_model = dpo_accelerator.unwrap_model(dpo_model)
unwrapped_dpo_model.save_pretrained('dpo_adapter')
dpo_tokenizer.save_pretrained('dpo_adapter')
print("successfully saved dpo adapter and tokenizer")
```

```
successfully saved dpo adapter and tokenizer
```

```
!cp -r dpo_adapter /content/drive/MyDrive/final-project/
```